12-06-23

Linked List

(M) capicity fixed. (overflow)

(*) shifting in insertion.

↳ takes time in removing / inserting

↳ can be considered as a drawback.

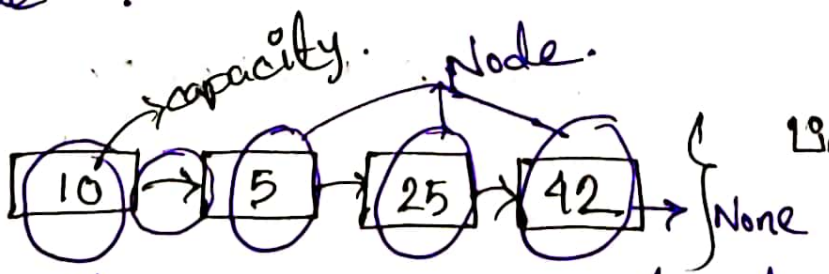
(**) shifting in deletion.

---

in order to solve these drawbacks of array, Linked list Data Structures is created.

(K) Every element in a linked list is a "node".

Ex:



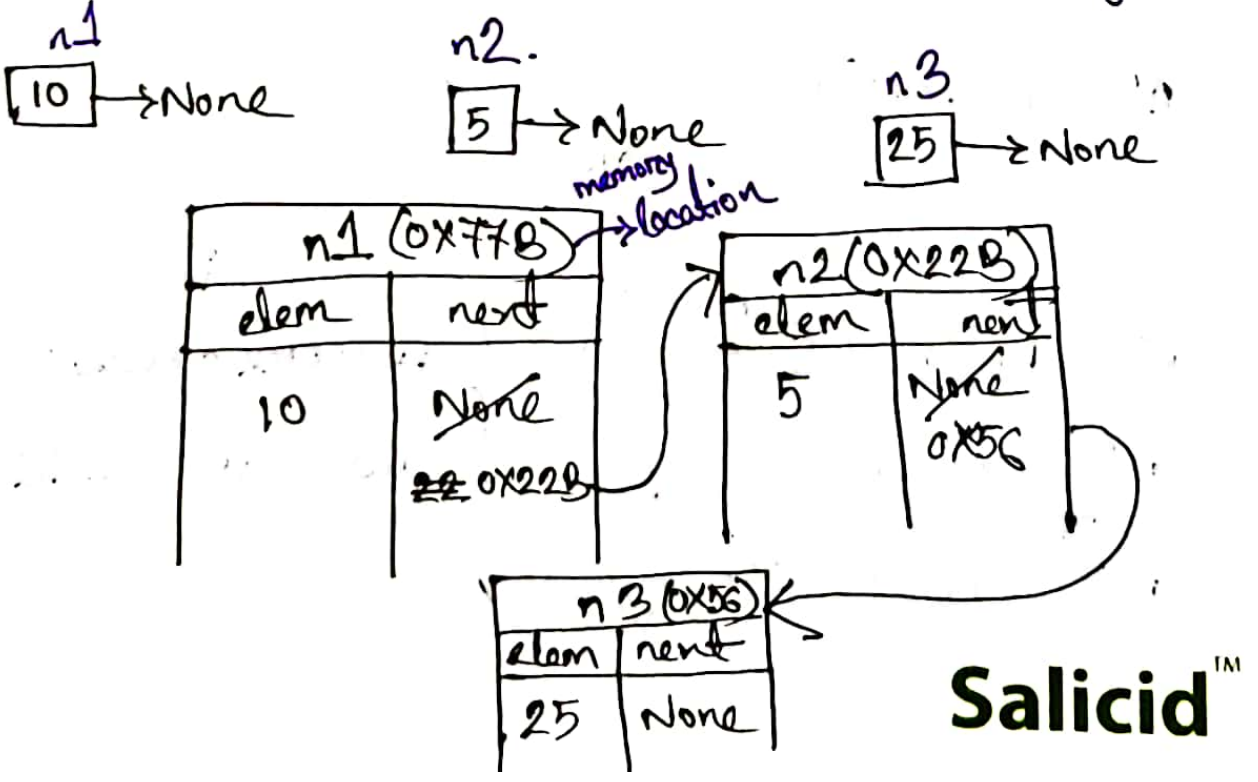
can add as many compartments we need

Code

```
class Node:
    def __init__(self, elem, next):
        self.elem = elem
        self.next = next

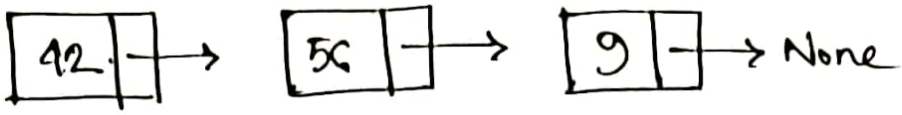
# Driver code:
n1 = Node(10, None)
n2 = Node(5, None)
n1.next = n2
n3 = Node(25, None)
n2.next = n3
print(n1) → this will print the memory location / reference.
```

Ex:



Salicid™

(*)




```
class Node :
    def __init__ (self, elem, next):
        self.elem = elem
        self.next = next
```

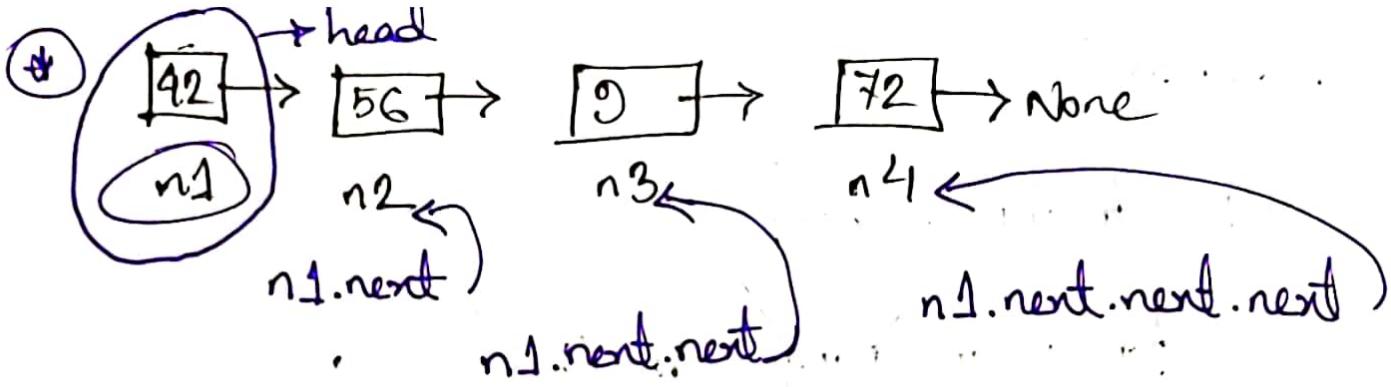
#

```
n1 = Node (42, None)
n2 = Node (56, None)
n1.next = n2.
n3 = Node (9, None)
A3 = n2.next
```

OR

```
n3 = Node (9, None)
n2 = Node (56, n3)
n1 = Node (42, n2)
```

} in ~~to~~ work ^ backwards and more efficient



* In Link list, backward iteration is not possible !

* A link list is given = the head of the link list is given.

Salicid™

Traversal:

```
def traverse(head):
    for i in ~~range~~ (len(head)):
```

node

* can't use "for" loop" and "len()" function in linked list.

For iteration:

i) head

ii) None

iii) head.next (to change it)

```
def traverse(head):
    while head != None:
        print(head.elem)
        head = head.next
```

→ condition line

→ iteration forward.

OR

```
temp = head
while temp != None:
    print(temp.elem)
    temp = temp.next
```

→ follow this format!

(focus on indentation)

```
#driver code:
n4 = ~~83~~.Node(72, None)
n3 = Node(9, n4)
n2 = Node(56, n3)
n1 = Node(42, n2)
head = n1
Traversal(head)
```

Simulation

head	Output	print
~~42~~	42	
~~56~~	56	
~~9~~	9	
~~72~~	72	
None		

Salicid™

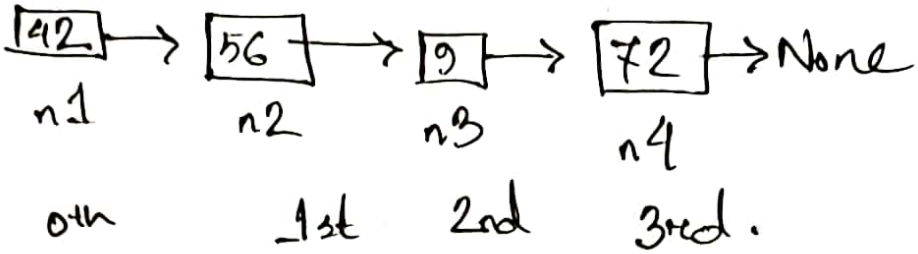
⑥ Count the nodes of a linked list:

```
def count_node (head):
    count = 0
    temp = head
    While temp != None:
        print (temp.elem)
        temp = temp.next
        count += 1
        print (count)
    return (count)
```

Driver code:

```
n4 = Node (72, None)
n3 = Node (9, n4)
n2 = Node (56, n3)
n1 = None (42, n2)
head = n1
Traversal (head)
c = count_node (head)
print (c)
```

(W)



Return the node of a link list in idx position:

```
def nodeAt (head, idx):
    temp = head
    count = 0
    idx = 0
    while temp != None:
        temp = temp.next
        count += 1
        if count == idx:
            return (idx)
        count += 1
```

Salicid™